

JDBC Type 1 to Type 4 Driver Migration

CO – 5 Assessment Tool - 2

Name : Tamil Elakiya S

Reg No : 192524443

Aim

To migrate a legacy Java application using the JDBC-ODBC Bridge (Type 1 JDBC driver) to a pure Java Type 4 JDBC driver and demonstrate how the migration simplifies application deployment.

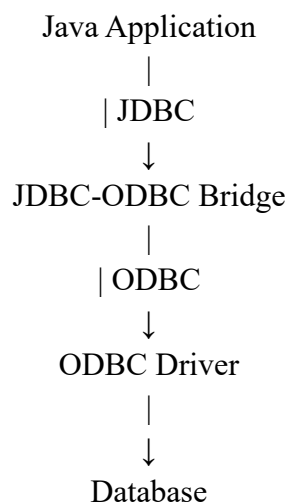
Problem Statement

A legacy Java application uses a JDBC-ODBC Bridge driver (Type 1) to connect to a database. The application needs to be migrated to a pure Java Type 4 JDBC driver. The migration should remove the dependency on ODBC configuration and provide a portable and simpler database connectivity solution.

Architectural Difference

Type 1 JDBC Driver – JDBC-ODBC Bridge

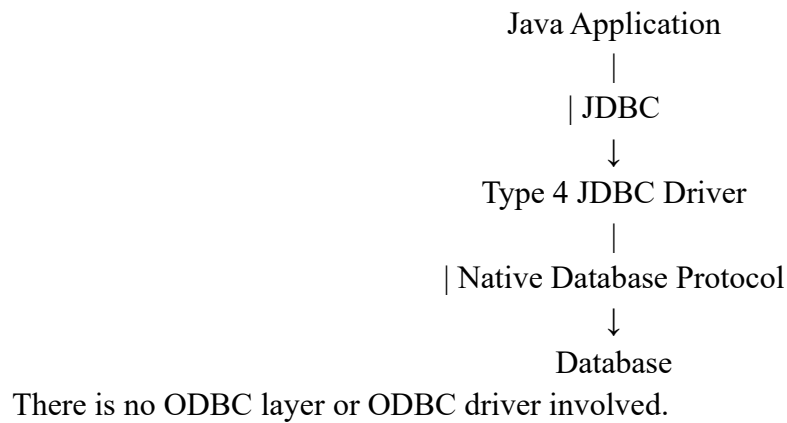
The Type 1 driver acts as a bridge between the JDBC API and an ODBC driver.



The Java application communicates with the database indirectly through the JDBC-ODBC bridge and ODBC driver.

Type 4 JDBC Driver – Pure Java Driver

The Type 4 driver is completely implemented in Java and communicates directly with the database using its native database protocol.



Comparison of Type 1 and Type 4 Drivers

Feature	Type 1 JDBC Driver	Type 4 JDBC Driver
Driver	JDBC-ODBC Bridge	Pure Java Driver
Communication	JDBC → ODBC → Database	JDBC → Database
ODBC Required	Yes	No
Implementation	Depends on Java and ODBC	Completely Java
Portability	Low	High
Deployment	Requires ODBC configuration	Requires only JDBC driver
Performance	Generally lower	Generally better
Modern Java Support	Obsolete	Recommended
DSN Configuration	Required	Not required

Deployment Problem in the Legacy Application

The Type 1 JDBC-ODBC Bridge requires an ODBC driver and an ODBC Data Source Name (DSN) to be configured on the system where the application is deployed.

Therefore, when the application is moved to another computer or server, the ODBC driver and DSN must be installed and configured again. This makes deployment operating-system dependent and increases configuration and maintenance requirements.

Also, the JDBC-ODBC Bridge was removed from the JDK starting with Java 8. Therefore, applications depending on it cannot be directly executed using modern Java versions.

Database Setup

The following MySQL database and table are used for demonstrating the Type 4 JDBC connection.

```
CREATE DATABASE assessment_db;
```

```
USE assessment_db;
```

```
CREATE TABLE students (  
id INT PRIMARY KEY AUTO_INCREMENT,  
name VARCHAR(100),  
department VARCHAR(100)  
);
```

```
INSERT INTO students (name, department)  
VALUES  
( 'Anu', 'Computer Science'),  
( 'Priya', 'Information Technology');
```

Type 4 JDBC Java Implementation

Code :

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
  
public class Type4JDBCExample {  
  
    private static final String URL =  
        "jdbc:mysql://localhost:3306/assessment_db";  
  
    private static final String USER = "root";  
  
    private static final String PASSWORD = "root";  
  
    public static void main(String[] args) {  
  
        try {  
            // Establish database connection  
            Connection connection =
```

```

        DriverManager.getConnection(URL, USER, PASSWORD);

System.out.println("Database connection successful!");

// Insert a student record
String insertSQL =
    "INSERT INTO students (name, department) VALUES (?, ?)";

PreparedStatement insertStatement =
    connection.prepareStatement(insertSQL);

insertStatement.setString(1, "Rahul");
insertStatement.setString(2, "Computer Applications");

insertStatement.executeUpdate();

System.out.println("Student inserted successfully.");

// Retrieve student records
String selectSQL =
    "SELECT id, name, department FROM students";

PreparedStatement selectStatement =
    connection.prepareStatement(selectSQL);

ResultSet resultSet =
    selectStatement.executeQuery();

System.out.println("\nStudent Records:");

while (resultSet.next()) {

    int id = resultSet.getInt("id");
    String name = resultSet.getString("name");
    String department =
        resultSet.getString("department");

    System.out.println(
        id + " | " + name + " | " + department
    );
}

// Close resources

```

```

        resultSet.close();
        selectStatement.close();
        insertStatement.close();
        connection.close();

        System.out.println("\nDatabase connection closed.");

    } catch (SQLException e) {

        System.out.println(
            "Database connection failed: "
            + e.getMessage()
        );
    }
}

```

Legacy Type 1 Code

The old Type 1 approach can be represented by the following legacy code:

```

import java.sql.Connection;
import java.sql.DriverManager;

public class LegacyJDBC {

    public static void main(String[] args) {

        try {

            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

            Connection connection =
                DriverManager.getConnection(
                    "jdbc:odbc:StudentDB"
                );

            System.out.println(
                "Connected using JDBC-ODBC Bridge"
            );

            connection.close();

        } catch (Exception e) {

```

```
        System.out.println(
            "Connection failed: " + e.getMessage()
        );
    }
}
```

Output

```
D:\JavaPrograms>javac Type4JDBC.java

D:\JavaPrograms>java -cp ".;mysql-connector-j-26.7.0.jar" Type4JDBC
Database connection successful!
Student inserted successfully.

Student Records:
1 | Anu | Computer Science
2 | Priya | Information Technology
3 | Rahul | Computer Applications
4 | Rahul | Computer Applications

Database connection closed.
```

Conclusion

The migration from a Type 1 JDBC-ODBC Bridge driver to a Type 4 pure Java JDBC driver changes the database architecture from an indirect connection to a direct connection.

The Type 1 architecture requires the JDBC-ODBC Bridge, an ODBC driver, and ODBC DSN configuration. This creates deployment and portability problems because every target system must have the required ODBC configuration.

The Type 4 driver communicates directly with the database using Java and the database's native protocol. Therefore, the ODBC dependency is completely removed. The application only requires the appropriate Type 4 JDBC driver and database connection configuration.

Thus, migrating to a Type 4 JDBC driver provides a simpler, more portable, maintainable, and modern database connectivity solution for Java applications.